

NOVACCÈS — Questions API pour brancher l'app agent

Pour : équipe backend `NovAcces.Api`

De : dev app agent (Expo React Native)

Mis à jour : 5 août 2026 (swagger relu ce jour : 80 opérations, 32 schémas)

En 30 secondes

L'app agent est **écrite et terminée**. Elle ne peut pas tourner : **on n'arrive pas à enrôler un terminal**, donc aucun écran au-delà du premier n'est atteignable.

#	Question	Ce que ça bloque	Priorité
Q1	Quel message exact faut-il signer pour <code>proofSignature</code> ?	Tout . Sans enrôlement, l'app s'arrête à l'écran 1.	● P0
Q2	Quel jeton met-on dans <code>Authorization: Bearer</code> sur <code>/api/agent/*</code> ?	Tous les appels agent	● P0
Q3	Que contient le QR visiteur (claims JWS) ?	Mode dégradé (REQ-SEC-06)	● P0
Q4–Q10	Forme des réponses, doublons d'endpoints, endpoints manquants	Fiabilité, cas limites	● P1–P2

Réponds d'abord à Q1. Une phrase suffit, du genre : « on signe `ticket + "." + deviceInstanceId` en ES256, signature en DER base64 ». Le reste peut suivre.

Deux remarques sur le swagger (valables sur les 80 opérations)

Ce ne sont pas des reproches, c'est juste pour expliquer pourquoi je pose autant de questions : **le contrat OpenAPI ne me dit rien sur les réponses ni sur l'authentification**.

- `components.securitySchemes` est vide et aucune opération ne déclare de paramètre d'en-tête (0 sur 80) — alors que tous les endpoints agent répondent `401 WWW-Authenticate: Bearer`.
- Aucune réponse n'est typée. Les 80 opérations déclarent `"200": { "description": "OK" }` sans corps. Aucun code d'erreur (`400` , `401` , `403` , `409` , `429`) n'est déclaré nulle part. Les corps de *requête*, eux, sont bien spécifiés.

Concrètement : je connais ce que j'envoie, jamais ce que je reçois. L'app lit donc les réponses « en devinant » (plusieurs graphies acceptées, échec explicite sinon). Chaque devinette est marquée **CONTRAT À CONFIRMER** dans `src/lib/api.ts`.

💡 Si c'est faisable de votre côté, **annoter les contrôleurs** (`[ProducesResponseType(typeof(XxxDto), 200)]` + `AddSecurityDefinition("Bearer", ...)`) répondrait d'un coup à Q2 et à toute la section P1. Ce serait de loin le plus rentable.

P0 — Les 3 bloquants

Q1. Preuve de possession de la clé du device

Ce que fait l'app : elle génère une paire ES256 sur l'appareil, garde la privée dans le Keystore/Keychain, et doit prouver qu'elle la possède en appelant :

```
POST /api/device-enrollments/activate
{ "ticket", "deviceInstanceId", "devicePublicKeyPem", "proofSignature" }
```

Ce qu'on observe en production :

Ce qu'on envoie	Ce que le serveur répond
deviceInstanceId non-GUID	400 { "error": "Identifiant du device invalide." }
GUID valide + proofSignature quelconque	400 { "error": "Preuve de possession de la clé du device absente ou invalide." }

La preuve est donc vérifiée **avant** le ticket. On a testé 5 hypothèses avec une vraie paire P-256 — **les 5 sont rejetées** :

Message signé	Encodage	Résultat
ticket	DER base64	✗
ticket	P1363 (r s) base64url	✗
deviceInstanceId	DER base64	✗
deviceInstanceId + "." + ticket	DER base64	✗
ticket + "." + deviceInstanceId	DER base64	✗

Ce dont j'ai besoin — merci de remplir :

```
Message signé : ..... (concaténation ? JSON canonique ? nonce du ticket ?)
Algorithme : ..... (ES256 / SHA-256 ?)
Encodage sig. : ..... (DER ou r||s brut · base64 ou base64url)
Format clé pub : ..... (SPKI "-----BEGIN PUBLIC KEY-----" ?)
Ticket : ..... (structure, durée de vie, contient-il un aléa à signer ?)
```

Le plus simple pour nous deux : collez le bout de C# qui vérifie la signature, ou un triplet d'exemple (ticket + clé + signature valide) qu'on puisse rejouer.

Q2. Authentification du terminal

Ce que fait l'app : après l'enrôlement, elle doit appeler `/api/agent/*` en `Authorization: Bearer ...`. On ne sait pas avec quoi.

Ce dont j'ai besoin :

- Que renvoie `POST /api/device-enrollments/activate` en cas de succès ? → **nom du champ** portant le jeton, **type** (JWT ou clé opaque), **durée de vie**, **comment le renouveler**.
 - Sur `/api/agent/*`, on présente **quel jeton** : celui de l'appareil, ou un jeton délivré par `shift/start` ?
 - **X-Site-Id** est-il **obligatoire** ? Le contrat PDF l'impose partout, le swagger ne le mentionne nulle part. Les deux ne peuvent pas avoir raison.
 - Quand un terminal sert **plusieurs sites** : `GET /api/agent/sites` existe, mais comment transmet-on ensuite le site retenu ? En-tête ? Paramètre ? État serveur lié au poste ?
-

Q3. Structure du QR visiteur

Ce que fait l'app : en mode dégradé (REQ-SEC-06), elle doit vérifier le QR **hors ligne**, pour qu'un QR forgé soit rejeté même pendant une coupure réseau. Elle a donc besoin de savoir lire le JWS elle-même.

Ce dont j'ai besoin :

- Les **claims** de la charge utile : `visitId` ? `exp` ? `siteId` ? autre chose ?
- L'**en-tête JWS** : valeur de `alg`, présence d'un `kid` ?
- La **correspondance des `kid`** avec ce que sert `GET /api/keys/public`.

Un seul QR d'exemple (chaîne compacte complète) répondrait aux trois d'un coup.

Pour info, c'est le seul endpoint dont on connaît la forme avec certitude, parce qu'il est public :

```
{ "kid": "current", "publicKeyPem": "-----BEGIN PUBLIC KEY-----\n...", "retiredKeys": [] }
```

P1 — Forme des réponses à confirmer

Pour chacun, il me faut juste le DTO renvoyé. Un copier-coller de la classe C# suffit.

Endpoint	Ce que j'ignore
POST /api/agent/shift/start	Jeton de session ? Identité de l'agent (nom, matricule) ? Identifiant de poste ?
GET /api/agent/sites	Tableau nu ou enveloppé ? Nom des champs identifiant / libellé
GET /api/site/config	Postes de contrôle (id + libellé) et paramètres (fenêtre -20/+15, TTL liste locale)
GET /api/agent/expected-today	Champs d'une visite attendue. Confirmer le moindre privilège : ni motif, ni coordonnées
GET /api/agent/offline-list	Voir Q8
POST /api/scan	Confirmer <code>ScanResponseDto = { isGranted, isCheckOut, isSecurityEvent, verdictCode, visitorName, overstayMinutes }</code>
POST /api/scan/manual-code	Voir Q10
POST /api/agent/resync	Forme du compte rendu : nombre d'acceptés + écarts ({ visitId, raison } ?)

Q4. L'énumération officielle des `verdictCode`. Elle n'existe aujourd'hui que dans le PDF. L'app affiche un écran plein format piloté par ce code — **un code inconnu ne doit jamais être présenté comme une autorisation**, donc la liste exhaustive est une exigence de sûreté, pas de confort.

Q5. Le format d'erreur. On observe `{ "error": "..." }`, non documenté. Et confirmer la sémantique de `409` (conflit de resynchronisation) et `429` (limitation de débit).

Q6. `direction` est un `string` libre dans `ScanRequestDto`. Valeurs confirmées côté source : `Entry` / `Exit`, insensible à la casse. Merci de le figer en `enum` dans le contrat.

Q7. `checkpointId` est présent dans le DTO mais son format n'est décrit nulle part. Doit-il correspondre à un identifiant issu de `GET /api/site/config` ?

P2 — Deux doublons à trancher

Q8. Deux endpoints de resynchronisation

Endpoint	Corps attendu
POST /api/agent/resync	<code>{ scans: [{ visitToken (uuid), direction, wasGranted, occurredAt, verdictCode, wasSecurityEvent, signedQrPayload }] }</code>
POST /api/scan/sync	<code>[{ signedQrPayload, direction, agentId, scannedAtUtc, offlineVerdict }]</code>

Lequel l'app agent doit-elle appeler ? Les modèles divergent vraiment : l'un identifie la visite par `visitToken`, l'autre par le payload signé seul ; l'un porte `agentId`, l'autre non. (L'app utilise `/api/agent/resync` aujourd'hui.)

Q9. Deux endpoints de liste hors ligne

GET `/api/agent/offline-list` (« Liste des QR valides du jour, signée, pour le mode dégradé ») et GET `/api/offline-list` (« Sert la liste quotidienne signée, avec un TTL de quatre heures maximum »).

- Lequel appeler ?
- La réponse est-elle un JWS compact brut, ou un objet JSON contenant un JWS ?
- Structure de la charge utile signée, et fréquence de rafraîchissement attendue ?

● P3 — Ce qui manque encore

Q10. Le code de secours : d'où vient-il ?

`POST /api/scan/manual-code { code, direction, checkpointId }` est apparu depuis notre dernier échange — merci, ça répond au cas « visiteur sans QR » (téléphone déchargé). Il reste à savoir :

- Où le visiteur obtient-il ce `code` ? `CreateVisitRequestDto` ne le contient pas, et la réponse de `POST /api/visits` n'est pas typée.
- Format et durée de vie du code ?
- Est-il utilisable hors ligne ? (a priori non, mais l'app doit le dire clairement à l'agent plutôt que d'échouer en silence)
- La réponse est-elle le même `ScanResponseDto` que `POST /api/scan` ?

Q11. Fin de poste — endpoint absent

`POST /api/agent/shift/start` ouvre un poste ; rien ne le ferme. Le terminal étant partagé entre agents successifs, le poste d'un agent parti reste ouvert — ce qui fausse l'imputation des scans dans le journal.

`POST /api/auth/logout` ne convient pas : il révoque un *refresh token* de compte web, pas un poste agent.

Demande : `POST /api/agent/shift/end`, idempotent.

Q12. Temps réel — hub SignalR

Le PDF mentionne `/hubs/scan-events` (réception de `VisitRevoked` / `VisitCreated` pour rafraîchir la liste locale sans attendre le prochain téléchargement). Il n'apparaît pas dans le swagger.

Existe-t-il ? Si oui : URL et contrat des messages. C'est optionnel, mais ça réduit la fenêtre pendant laquelle un QR révoqué reste accepté hors ligne.

Récapitulatif

Bloquant — l'app ne démarre pas sans ça :

- ☐ Q1 Message exact signé pour `proofSignature` (+ *algo, encodage, format clé*)
- ☐ Q2 Jeton `Bearer` sur `/api/agent/*` · `X-Site-Id` obligatoire ou non
- ☐ Q3 Claims + en-tête du QR JWS

Important :

- ☐ Q4 Liste exhaustive des `verdictCode`
- ☐ Q5–Q7 Format d'erreur · `direction` en enum · format de `checkpointId`
- ☐ P1 DTO de réponse des 8 endpoints du tableau

À trancher :

- ☐ Q8 `/api/agent/resync` ou `/api/scan/sync` ?
- ☐ Q9 `/api/agent/offline-list` ou `/api/offline-list` ? JWS brut ou enveloppé ?

Manquant :

- ☐ Q10 Origine, format et durée de vie du code de secours
- ☐ Q11 `POST /api/agent/shift/end`
- ☐ Q12 Hub SignalR : existe-t-il ?

Base de l'analyse : swagger <https://api.sigasaccs.com/swagger/v1/swagger.json> (relu le 5 août 2026 — 80 opérations, 32 schémas, 0 réponse typée) · contrat fonctionnel [docs/contrat-api-app-agent.pdf](#) · sondages sur l'API de production les 3 et 5 août 2026.